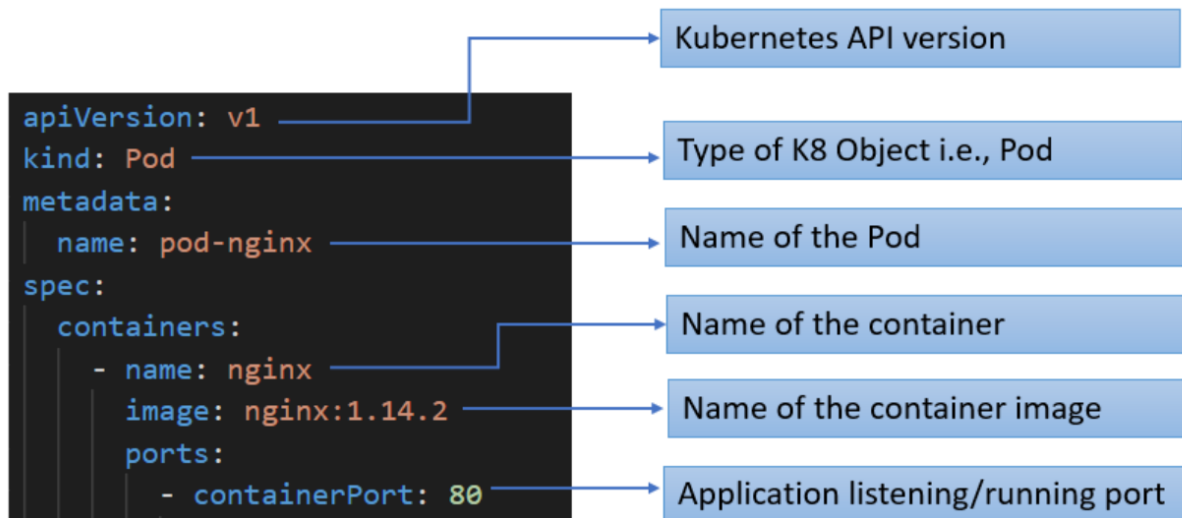


Note:

Example pod-manifest file



How to login into a running container in docker?

docker ==> docker attach <container-name>

(or)

docker exec -it <containername> /bin/bash

Note: /bin/bash command will open linux terminal inside the container

How to login into a pod(container)?

kubectl exec -it <podname> -- <linux_command>

kubectl exec -it mytomcatpod1 -- /bin/bash

Note: /bin/bash command will open linux terminal inside the pod / container

to come out of pod use ==> exit (this will not stop container as pod always runs containers in detached mode)

How to run any command in pod from k8s master?

kubectl exec -it <pod_name> -- <linux_command>

eg: to get container os information

kubectl exec -it <pod_name> -- cat /etc/os-release

To create environment variables in pod manifest files:

syntax in spec block is:

```
spec:
  containers:
  - name: <container-name>
    image: <image-name>
    env:
    - name: <variable_name>
      value: <variable_value>
```

+++++

To create a pod with a single container.

Example 4: Create a pod with below configurations

- pod name as **mytomcatpod** , image **tomee** & container **tomee-container3**
- labels ==> **type: web-server-tool** &
- environment variables ==> **myusername** as **devopsadmin**

```
---
apiVersion: v1
kind: Pod
metadata:
  name: tomcat-pod
  labels:
    app: my-app
spec:
  containers:
  - name: c1
    image: tomcat
    env:
    - name: myusername
      value: devopsadmin
    - name: password
      value: mypassword
...
```

kubectl apply -f <manifest-file>.yaml

to see env variables details in k8s master node use ==>

kubectl describe pod <pod_name> (observe env variables defined here)

to see env variables details by logging inside the pod/container

```
kubect1 exec -it <pod_name> -- /bin/bash
echo $myusername (from output - observe variable substitutions)
echo $password (from output - observe variable substitutions)
```

Port Mapping:

Syntax / Template to write pod manifest file with port mapping

```
apiVersion: v1
kind: Pod
metadata:
  name: <Name-of-Pod>
  labels:
    <label-name>: <label-value>
spec:
  containers:
  - name: <Name-Of-Container>
    image: <image-Name>
    ports:
    - containerPort: <port-Num-Container>
      hostPort: <host-port-number>
```

+++++

Example 6: Create a pod with below configurations

- pod name as **apachetomcatpod**, image **tomcat:8-jre11** & containername **c1**
- labels ==> **type: my-app**
- **application** runs on **port 8080** & map it to **hostport** (worker-node) on **31111**

```
---
apiVersion: v1
kind: Pod
metadata:
  name: apachetomcatpod
  labels:
    app: my-app
spec:
  containers:
  - name: c1
    image: tomcat:8-jre11
    ports:
    - containerPort: 8080
      hostPort: 31111
...
```

Commands recap:

To create a object with a manifest file `kubectl create -f <object_file_name>.yaml`

To list all the pods `kubectl get pods`

To display more information of pods `kubectl get pods -o wide` (here: -o means output)

Generic kubernetes commands that can be used across all Kubernetes objects(pods, replicaset, deployments ,services etc):

To display list of the <object_name> in the cluster `kubectl get <object_name>`

To display more information of <object_name> `kubectl get <object_name> -o wide`

To see the information about a particular object `kubectl describe <object_kind> <object_name>`

To delete a k8s object `kubectl delete <object_kind> <object_name>`

Note:

pod is basic object which we can create in k8s, similar to pods kubernetes also have other objects like replicaset, replication-controller , deployments , services etc...

+++++

***** important interview questions *****

Q1. What are labels in k8s & why do we need labels?

- labels are user provided key-value pairs, using labels we can **organize**(group) Kubernetes objects
- using labels, we can **filter** different Kubernetes objects
- Same label (key/value) can be assigned to multiple Kubernetes objects

To filter based on based on labels

syntax: `$ kubectl get pods -l <labels-key>=<labels-value>`

if we have multiple pods in our cluster & if we want to list only Pods which have label as ==> app:nginx

`$ kubectl get pods -l app=nginx`

Q2: What are selectors?

- Selectors helps to find the k8s objects based on their labels & uses it.

Q3: Why do we need to use labels & selectors?

- By using labels and selectors, it will be easier to manage complex applications in Kubernetes.

Controller- manager:

Controller manager ==> (actual state == desired state)

Controller manager is the brain behind the orchestration, it always monitors actual state of objects with that of desired states.

How Controller Manager Works?

1. **Observes** the cluster if there are any changes.
2. **Compares** the current(actual) state with the desired state.
3. **Takes action** (creates, deletes, or updates k8s objects) to achieved desired state.

Controller manager objects

1. Replication-controller --> older feature (not used now a days)
2. Replicasets controller --> newer feature
3. Deployment controller.
4. Deamonsets controller.
5. Statefullsets controller.
6. Node Controller

ReplicaSet:

- ReplicaSet is a k8s object that **ensures a specified number of replica Pods** are **always running** .
- If a Pod managed by a ReplicaSet fails or is deleted, the ReplicaSet will **automatically create a new replica** to replace it
- using ReplicaSets we can achieve high availability, load balancing and scaling.

Why do we need to use ReplicaSet ?

Suppose if we deploy only one pod of the application and by any chance if it fails after creation, then the application is no longer accessible to the user.

ReplicaSet is nothing but the additional copies/replicas of the pod.

it will keep the desired instances of pods always running

Note on replicaset manifest file:

- ReplicaSet uses elements like "**replicas**", "**selector**" & "**template**" field in its '**spec**' section.
- Template contains all pod related information so it can be also called as pod template
- copy pod manifes file contents ==> paste it under "template" ==> without apiversion & kind

SYNTAX of ReplicaSet manifest file:

```

---
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: <replicaSetName>
spec:
  replicas: <noOfReplicas>
  selector:
    matchLabels:
      <key>: <value>
  template: # POD Template
    metadata:
      name: <PODName>
      labels:
        <key>: <value>
    spec:
      containers:
      - name: <nameOfTheContainer>
        image: <imageName>
...

```

Example: create a replicaset myreplicaset-1 with 5 replicas with tomee image



```

1  ---
2  #Pod manifest file
3  apiVersion: v1
4  kind: Pod
5  metadata:
6    name: my-pod-11
7    labels:
8      author: bharath
9  spec:
10   containers:
11     - name: c1
12       image: tomee
13
14
15

```

copy & paste

```

1  #create a rs with 5 replicas of tomee image
2  apiVersion: apps/v1
3  kind: ReplicaSet
4  metadata:
5    name: my-replicaset-1
6  spec:
7    replicas: 5 #Number of pods
8    selector:
9      matchLabels:
10        author: bharath
11    template: #pod manifest without apiversion & kind
12      metadata:
13        name: my-pod-11
14        labels:
15          author: bharath
16      spec:
17        containers:
18          - name: c1
19            image: tomee

```